FEDERAL UNIVERSITY OF RIO DE JANEIRO
POLYTECHNIC SCHOOL
DEPARTMENT OF ELECTRONIC AND COMPUTER ENGINEERING

UNDERGRADUATE PROJECT PROPOSAL

Student: Matheus Hoffmann Fernandes Santos
hoffmann@poli.ufrj.br

Advisor: To be defined, Dr.
Co-advisor: Pedro Veras Guimarães, Dr. (LabECO, Department of Mechanical Engineering, UFSC)

1. TITLE

Operational optimisation of the WAVEWATCH III® wave model for GPU.

2. EMPHASIS

Computing / High-performance scientific computing.

3. THEME

WAVEWATCH III® (WW3) is a numerical model for forecasting ocean waves, developed by the National Centers for Environmental Prediction (NCEP) of the US National Oceanic and Atmospheric Administration (NOAA) and distributed as open source (NOAA-EMC, 2026a; The WAVEWATCH III Development Group, 2019). The model represents the ocean as a grid of points and describes the sea state at each point by the wave spectrum, the distribution of wave energy over frequencies and directions. At every time step this energy is propagated across the grid and modified by the so-called source terms: the energy the wind transfers to the waves, the energy exchanged among the waves themselves, and the energy lost to breaking. WW3 belongs to the class of third-generation models, which solve the wave action balance equation (action being energy divided by frequency) without imposing the shape of the spectrum in advance (The WAMDI Group, 1988; Tolman, 1991). The code is written in Fortran and parallelised with MPI and OpenMP, and a *switch file* selects at compile time which parts of the code enter the executable, such as the physics, the propagation scheme and the I/O options (The WAVEWATCH III Development Group, 2019). In Brazil, the Laboratório de Engenharia e Ciências

Oceânicas of the Universidade Federal de Santa Catarina (LabECO/UFSC) runs WW3 in its operational forecasting system, as part of the Rede Nacional de Observação e Monitoramento Oceânico (ReNOMO) (LabECO/UFSC, 2026; UFSC, 2026).

The theme of this project is the **parallel simulation and GPU acceleration of a spectral wave model**, taking WW3 in LabECO's operational configuration as the case study. The question behind the work is how much of the wall-clock time of a forecast cycle can be cut without the forecast fields differing from the current ones by more than the tolerances agreed with the laboratory, and whether a GPU helps when the model state stays in CPU memory. The work starts from two hypotheses, both testable by measurement. The first is that the current run time can still be reduced through build and configuration changes alone, by a margin larger than the variation between repeated runs. The second is that, even with the model state kept on the CPU, running the most expensive routines on a GPU reduces the total cycle time. That only happens if the compute time saved exceeds the cost of moving data between CPU and GPU at every time step, a cost that depends on the volume copied and on the CPU–GPU link in the laboratory's node.

4. SCOPE

The work is limited to WW3's single-grid program (`ww3_shel`). The model's pre- and post-processors, such as `ww3_grid` and `ww3_ounf`, are used as they are, and only their input files may change. The test cases, on regular and unstructured grids, come from LabECO's operational forecasting system, whose configuration the laboratory will hand over at the start of the schedule. Unstructured grids are triangle meshes that allow finer resolution near the coast; in parallel runs, WW3 splits them among processes with the PDLIB library (Parallel Decomposition Library). Each case consists of the grid, the switch file, the forcing (the external fields that drive the model, mainly wind) and the output list. At the start of the project, the configuration of each case, as it runs at the laboratory today, and the exact source revision will be recorded and kept fixed (frozen) until the end of the work, so that every measurement always refers to the same cases. WW3's official regression tests serve only as a validation instrument.

Four rungs are in scope, carried out in this order and each validated against the reference run:

1. **Build options.** The compiler, the optimisation flags and the layout of MPI processes and OpenMP threads will be chosen. In the switch file, forcing interpolation and the parallelisation mode will be revisited: shared memory, MPI or hybrid and, on the unstructured grid, domain decomposition through PDLIB. This rung does not change the source code.
2. **Run configuration.** The time steps will be tuned within the numerical stability limit given by the Courant–Friedrichs–Lewy (CFL) condition, which, for explicit schemes, relates the time step, the grid spacing and the propagation speed of the waves. The output frequency and field list and the restart strategy will also be revisited. This rung changes only the namelists, the parameter files read by WW3's programs.
3. **Targeted modern-Fortran refactoring, assisted by a large language model (LLM).** The routines the performance profile shows as dominant, usually the source-term integration (`W3SRCE`) and propagation on the regular grid, will be rewritten using Fortran 2008/2018 constructs (`do concurrent`, explicit interfaces, contiguous arrays) without changing the numerical algorithm. An LLM-based coding assistant will propose the rewrite, routine by routine, as in the work of Koldunov et al. (2026) on FESOM2, an ocean circulation model.
4. **Rewriting kernels in C++/Kokkos and running them on a GPU (NVIDIA H100).** Kokkos is a C++ performance-portability library that compiles the same source code for CPUs and for GPUs from different vendors (Trott et al., 2022). The routines that remain dominant after rung 3 will be rewritten, one at a time, as Kokkos kernels called from Fortran. If the nonlinear interaction term, which represents the energy exchange between waves of different frequencies, is still among them, it will be the first, because it is self-contained and easy to test on a synthetic spectrum. The project does not plan to move the whole model to the GPU: the model state will stay in Fortran, in CPU memory, and the cost of moving data between CPU and GPU at every time step will be measured and reported.

The order of the rungs is part of the scope. Rungs 1 and 2 will be exhausted

before any code change, and rung 3 before rung 4, so that the C++/Kokkos rewrite is only done on routines whose cost after the earlier rungs has been measured. Source terms are computed point by point and are therefore the same on both grid types; in rungs 3 and 4 they will be handled on both. Propagation on the unstructured grid uses its own routines, tied to PDLIB, and will be kept as it is.

In parallel, NOAA is developing WAVEWATCH IV (WW4), a rewrite of the model with a C++ core (NOAA-EMC, 2026b; Tolman, 2025, 2026). This project is independent of that effort and does not intend to contribute code to it. From WW4 it adopts only the test structure (unit and integration levels, in GoogleTest) for the rewritten kernels, so that they can be reused in a future migration.

Out of scope are the multi-grid program (`ww3_multi`), coupling with other models, data assimilation, and any change to the physics parameterisations, that is, to the formulations of the source terms.

5. JUSTIFICATION

Wave models are driven by the wind forecast by atmospheric models. Because those models are governed by nonlinear equations, the wind forecast, and with it the wave forecast, loses quality as the forecast horizon grows. Campos et al. (2020) compared the NCEP ensemble forecast system with measurements from four altimeter satellites, which record wave height from space, and showed that the error grows along the forecast days. For that reason, operational systems update the forecast several times a day and use ensemble forecasting, in which the model is run many times with small perturbations of the initial conditions and the forcing. The mean of the runs tends to err less than a single run (Farina, 2002). In the NCEP system, the ensemble mean cut the random component of the error in significant wave height, which roughly corresponds to the mean of the highest third of the waves, from 1.8 m in the control run (no perturbations) to 1.3 m (Campos et al., 2020).

Cycle frequency, resolution, and the number of nested grids and of ensemble members are bounded by the run time of each forecast and by the computing resources available. NCEP, for instance, runs GFS-Wave, its global wave forecast system, four times a day (00, 06, 12 and 18 UTC) on global grids of 0.16° and 0.25° with finer regional grids nested inside, plus GEFS-Wave, the ensemble version, with

30 members at 0.25°, both out to 384 h (NOAA/NWS/NCEP/EMC, 2026).

The same holds for LabECO. ReNOMO, funded by the CNPq/MCTI/Finep 062/2022 call, integrates ocean observation, quality control and operational products for the South Atlantic and the Brazilian coast (LabECO/UFSC, 2026). For the wave field that feeds those products, a shorter cycle allows earlier delivery, a finer grid at the same deadline, or ensemble members to quantify uncertainty.

WW3 was designed for the supercomputers of two decades ago, and NOAA itself acknowledges that the model's parallel concepts and data structures, inherited from the transition to Fortran 90, have not kept up with the hardware (Tolman, 2025). High-performance computing is moving from general-purpose processors to heterogeneous nodes with many cores and accelerators such as GPUs. That shift has led weather, ocean and climate prediction codes to revisit their structure (Bauer et al., 2021) and motivated performance-portability layers such as Kokkos (Trott et al., 2022).

Rungs 1 and 2 come first because they require no change to the code. Moreover, according to the model's manual, part of the performance depends on choices made outside the source code. The switch file and the compiler options determine which code is compiled and how, and the layout of MPI processes and OpenMP threads determines how much time is spent exchanging spectra between processes rather than computing them (The WAVEWATCH III Development Group, 2019). In an installation configured once, when the model was deployed, and not revisited since, part of the run time may be due to the configuration rather than to the code. These rungs cost little, only machine hours, and produce the two instruments the later rungs depend on: a reproducible benchmark, which measures run time the same way every time, and a performance profile, which shows in which routines that time is spent. Without these, there is no way to know how much time can still be saved, or in which routines. Code will therefore be rewritten only after the benchmark and the profile exist.

Rung 3 is justified by the structure of the code. The profile published by Ikuyajolu et al. (2023) shows that the source-term integration dominates WW3's run time, with propagation second, and that the routines involved use large local arrays and global variables declared in modules, which hinders both vectorisation

and execution on a GPU. Rewriting these routines in modern Fortran, preserving the algorithm and the arithmetic, is engineering work of known risk, with directly measurable gain in the model the laboratory operates. It also prepares the next rung, since a routine with an explicit interface and contiguous data in memory can be replaced by a kernel.

Rung 4 answers a question from the laboratory itself: whether an H100 GPU can accelerate the model in operation. Upstream WW3 offers no GPU execution (NOAA-EMC, 2026a). The only published port on record moved the source terms to the GPU with OpenACC directives and obtained a speed-up of about 1.3 times per node, limited by data transfer between CPU and GPU at every time step (Ikuyajolu et al., 2023). By contrast, the authors of WAM6-GPU, the GPU version of another third-generation wave model, restructured the whole model to keep all fields in GPU memory and obtained an order-of-magnitude gain on a multi-GPU node (Yuan et al., 2024). The two results indicate that the gain depends on the data staying on the GPU: when the model state lives on the CPU and must be copied at every step, the transfer limits the gain. Even so, this project will keep the state in Fortran, for three reasons. Moving the whole model does not fit the timeline of an undergraduate project; replacing one routine at a time allows each kernel to be validated in isolation, by comparison with the original code; and measuring the cost of that transfer on the H100 is itself a useful result for the laboratory. What may make this project's result differ from that of Ikuyajolu et al. (2023) is the volume copied per call, which rung 3 will reduce by making each routine's inputs and outputs explicit instead of module-level global variables. Even so, the outcome of rung 4 may be negative, that is, a kernel may become faster on the GPU without the cycle getting shorter. The translation will follow the method of Koldunov et al. (2026), who translated some 74 thousand lines of FESOM2 Fortran to C and then to C++/Kokkos in weeks, with an expert-directed LLM coding assistant, two-stage literal translation and validation in successive levels. That work has not yet been peer-reviewed. The criterion for taking a kernel into operation will be the same as for the other rungs: measured gain and results within tolerance.

State of WW3 testing. The proposed validation rests on the model's own test infrastructure, as it stood on the `develop` branch (version 7.14) in September

2026 (NOAA-EMC, 2026a). WW3 has 62 regression test cases in `regtests/`, run by a matrix of scripts that compiles switch combinations, compares the outputs of two runs file by file with `cmp` and `diff`, and classifies each case as identical or not. The matrix includes variants that check bit-for-bit reproducibility after a restart and with different numbers of threads and of processes. There is no tolerance-based comparison: any numerical difference is recorded as a failure, and it falls to the reviewer to judge whether it is acceptable. The unit tests, introduced in 2023, cover input and output only (five programs, via CTest). That picture defines how the project will validate each rung. In rungs 1 and 2, the model's own matrix will validate the options that do not change the arithmetic; these must reproduce the reference bit for bit. Options that change the solution require a field-by-field comparator with tolerances, and rungs 3 and 4 also require per-routine unit tests. WW3 has neither, and the project will build both.

State of WW4. The WAVEWATCH IV repository was created in November 2025 and, on 7 October 2026, still had no published release (NOAA-EMC, 2026b). Phase II of the project, which dealt with languages, governance and architecture, was completed in March 2026 (Tolman, 2026). On 7 October 2026, the C++ core still had no source terms or propagation; a change under review would only add the selection, at run time and without recompiling, of the propagation scheme and of the source terms, none of them implemented yet (NOAA-EMC, 2026b). On that date, the test strategy defined four levels in GoogleTest, of which only the first two existed, unit (L1) and integration (L2); the functional (L3) and regression (L4) levels, which would replace the WW3 matrix, had not been written (NOAA-EMC, 2026b). The discussion of the CPU–GPU architecture, which proposed Kokkos as the data abstraction layer, was closed without a recorded decision, and no file in the code mentioned Kokkos (NOAA-EMC, 2026b). According to the Phase I report, the first public release was expected in mid-2027, and most WW3 support will only end once WW4 is mature (Tolman, 2025). Two consequences follow: WW3 will remain the laboratory's operational model for some years, which justifies optimising it, and the kernels rewritten here, in Kokkos and with L1 and L2 tests in the WW4 style, can serve as evidence for WW4's CPU–GPU architecture choice without duplicating the model's development.

6. OBJECTIVE

The general objective is to reduce the run time of the operational WAVEWATCH III cycle maintained by LabECO/UFSC for ReNOMO, with measured and reproducible gains, without the results differing from the current ones by more than the tolerances agreed with the laboratory. A single criterion (measured gain and results within tolerance) will also decide whether C++/Kokkos kernels running on an NVIDIA H100 GPU become part of the operational configuration.

The specific objectives are:

1. To record and freeze the operational configuration of each case (source revision, switches, namelists, grid, forcing, outputs and hardware) and to build a reproducible benchmark of the reference run, with a defined metric: run time per forecast hour.
2. To profile the reference run by routine and by phase (source terms, propagation, communication, input and output), on one and on several MPI processes.
3. To quantify the gain of the build, configuration and Fortran refactoring rungs, each with its check of agreement with the reference, and to deliver the best configuration to the laboratory as a documented build.
4. To build the validation infrastructure WW3 lacks: a field-by-field comparator with versioned tolerances and per-routine unit tests on inputs captured from the operational case, integrated with the model's regression matrix.
5. To rewrite in C++/Kokkos, in the order given by the profile, the routines that remain dominant, validate them on CPU against the original Fortran, measure the gain on GPU (H100) and decide, on gain and agreement of results, whether they enter the operational configuration.
6. To publish tooling, results and recommendations in the project repository, in a form that lets the laboratory repeat the measurements, subject to the disclosure policy described in the methodology.

7. METHODOLOGY

The project will follow the principle of measuring before optimising and will advance in test-and-validation cycles. Two quantities will be measured in every cycle: run time, total and per phase of the time loop, expressed as `ww3_shel` run

time per forecast hour, with post-processing recorded separately; and agreement of the results, assessed by the difference between the fields produced by the changed run and by the reference run. Each timing will be repeated at least five times on a dedicated node, and the report will give the median and the interquartile range. In each cycle a single element will be changed (an option, a parameter or a routine), and the tests will run from the cheapest to the most expensive: first the isolated routine, when code has changed, then a simplified grid and, last, the operational case. The results will then be compared with the reference; from rung 4 onwards, the project will also compare the CPU run and the GPU run of the same code on the simplified grid. A change that passes will be incorporated and become the baseline of the next cycle, and one that fails will go back to rewriting or be discarded.

Reference cases and simplified grids. The laboratory's operational cases, on a regular and on an unstructured grid, will be captured as self-contained tests, with input files, forcing for a fixed period, switch file, namelists and reference results. For each grid type a simplified grid will also be defined, a reduced domain derived from the operational case or from the closest official regression test, which runs in a few minutes on both CPU and GPU and will be used in every test cycle. The simplified grid only speeds up the cycles and does not replace the operational case in any reported measurement. The benchmark script will record time per phase, the number of processes and threads, the compiler and its options and the source revision, and will produce a table that can be attached to the reports.

Profiling. The performance profile of the reference run will be taken with two complementary tools. `gprof` instruments the executable at compile time (the `-pg` option) and produces the call graph, with the time accumulated in each routine and in the routines it calls (Graham; Kessler; McKusick, 1982). `perf`, distributed with the Linux kernel, periodically samples the run through the processor's performance counters, without recompiling the model, and attributes time to routines and source lines (Linux Kernel Organization, 2026). The former gives the call structure, and the latter confirms the timings without the interference of instrumentation. Measurements will be taken on one, four and sixteen MPI processes, with communication time recorded separately. The result will be a time-per-routine table, grouped by the phases of the time loop, which will set the order of the refactoring and serve

as the baseline for every later measurement.

Rungs 1 and 2. In rung 1, build options will be explored as a matrix of compiler, options and parallel layout, which includes, on the unstructured grid, building with PDLIB. In rung 2, the starting point will be a single set of namelists per case, the operational one, frozen at the start of the project and also used in every other rung. From it, the project will vary, one at a time, only the parameters listed in the scope, which affect performance without changing the physics parameterisations. The outcome will be at most a second set of namelists, documented and proposed to the laboratory.

Rungs 3 and 4. In rung 3, the Fortran refactoring will address the routines at the top of the profile, one at a time, in a fork of the code that keeps the original routine in the same executable, selectable at run time. Only then will rung 4 begin, restricted to the routines the updated profile still shows as dominant.

Agreement criteria and order of the tests. In rungs 1 and 2, WW3's own regression matrix will be used as it is. Options that do not change the model's arithmetic, such as the layout of processes and threads and flags that, with the same compiler, keep floating-point semantics, must reproduce the reference bit for bit, file by file, on the operational cases and the simplified grids. Those that change the solution or the files will be assessed by the field-by-field comparator described next, on the fields and times common to both runs. That group includes a change of compiler, flags that reorder operations or fuse a multiplication and an addition into one instruction (FMA, fused multiply-add), forcing interpolation, the time step, the output frequency and the output list. The tolerances of that comparator will be set by the LabECO research group for each field, such as significant wave height, peak period and peak direction, which are the period and direction of the waves that carry the most energy. In rungs 3 and 4, where the code changes, the tests will follow two levels, in this order. First, before any run of the full model, per-routine unit tests, which the matrix does not offer, will run the original and the rewritten routine on inputs captured from the operational case and compare the output arrays in memory, which detects an error in seconds and already points to the routine responsible. Then the field-by-field comparator will read the NetCDF outputs of two full runs, first on the simplified grid and then on the operational

case, and apply tolerances recorded in a versioned file. Since the NetCDF file is the final product of the chain, this test comes last. A change that fails at any level will not be incorporated: it will go back to rewriting and repeat the cycle from the unit tests.

Kernel interoperability and testing. Each C++/Kokkos kernel will be called from Fortran through an `ISO_C_BINDING`/`bind(C)` interface, and the original routine will stay in the executable, selectable by the same option, so that the regression matrix and the comparator can test both paths without recompiling. The layout of the Kokkos `Views` will reproduce that of WW3's spectral arrays, so that the CPU call copies no data. The translation will follow the method applied to FESOM2 (Koldunov et al., 2026). First, a literal translation to C will be made and checked against the Fortran on captured inputs. Next, a Kokkos version will be written, whose Serial backend must reproduce the C bit for bit. Only then will the CUDA backend run on the H100 and be compared with the CPU run through the maximum error and the root-mean-square error (RMSE) of each field, first on the simplified grid and then on the operational case, with the volume of data moved between CPU and GPU at every time step recorded. The tests will follow the WW4 structure. At level L1, each kernel will have a GoogleTest unit test built from a synthetic wind-sea spectrum whose empirical shape was obtained in the JONSWAP project (Joint North Sea Wave Project) and described by Hasselmann et al. (1973), with declared and justified tolerances. At level L2, the simplified grid and the operational case will be rerun, and the field-by-field comparator will compare the original WW3 with WW3 with Kokkos. The decision to take a kernel into operation will be made with LabECO on two pieces of evidence: the timing table of the operational case, with and without the kernel, and the agreement report.

LLM-assisted activities. An LLM-based coding assistant, directed by the student, will be used in four activities: reading and documenting WW3's dominant routines, to map their dependencies on global variables before the rewrite; proposing the modern-Fortran refactoring, routine by routine, in rung 3; the literal translation from Fortran to C and from C to C++/Kokkos in rung 4, following the method of Koldunov et al. (2026); and generating the unit tests and the scripts that capture inputs and compare outputs. No code generated by the assistant will be accepted

without meeting the same agreement criteria required of hand-written code, and the student and the co-advisor will be responsible for the code review and for the agreement evidence. Prompts, the reviewed diff and the agreement evidence will be versioned with the code. The final report will record, per activity, what the assistant produced, what was corrected by hand and what was discarded, as an assessment of the method itself.

Tooling and reproducibility. The code, the tooling and the tests will be versioned in the public `ww3-gpu` repository (Hoffmann, 2026), which provides the pinned toolchain, the build and regression scripts, the model as a submodule at a pinned revision, and the continuous integration, so that any measurement can be repeated by the laboratory on its own machines. The operational configuration and LabECO's data, as well as results not yet published, will stay outside the public repository, in a restricted location set by the laboratory. These data and results will remain restricted until the defence of the undergraduate project or the submission of a paper by the research group, whichever comes first, unless the advisors decide otherwise. Tests that depend on inputs captured from the operational case will live with those data; the public continuous integration will use only the official regression tests and synthetic spectra. Results will be recorded as tables and figures generated by scripts.

Deliverables and validation. The deliverables are the configuration record, the benchmark and profile tables, the best documented build, the field-by-field comparator and the unit tests, the Kokkos kernels with their agreement and timing evidence, and the final report with the operational recommendation. Scientific validation will rest with the LabECO research group, which sets the tolerances and reviews the agreement evidence; engineering validation, with the repository's automated tests and independent reruns at the laboratory.

Risks. The main risk is a divergence of results that goes unnoticed after a rewrite, mitigated by the agreement criteria and the per-routine tests. The second is rung 4 consuming the time of the earlier ones, mitigated by the fixed order of the rungs and by the rule of rewriting only routines whose cost after the earlier rungs has been measured. The third is the GPU gain being bounded by data transfer while the state remains in Fortran. In that case, the result will be the measurement

of that bound, a recommendation not to run on the GPU in that configuration, and the next piece of work, which is keeping the data on the GPU from one time step to the next. The fourth is a delay in receiving LabECO's cases or in access to the H100. In that case, rungs 1 and 2 will start provisionally with the official regression test closest to the operational configuration, and those numbers will not be reported as results of those rungs. The rung 4 measurements will be taken on the GPU available, with the caveat that they do not carry over directly to the H100.

8. SCHEDULE

The project will take two academic periods, and the dates will be adjusted with the advisors.

Table 1: Undergraduate project schedule.

	2026			2027				
Activity	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May
Bibliographic review; configuration record of the operational cases (regular and unstructured grids)	▬							
Reproducible benchmark, simplified grids, field-by-field comparator and performance profile		▬						
Rung 1: build options (compiler, flags, switch file, parallelisation, PDLIB)			▬					
Rung 2: run configuration; per-routine unit tests				▬				
Rung 3: modern-Fortran refactoring of the dominant routines					▬			
Rung 4: C++/Kokkos kernels; agreement on CPU and measurement on the H100						▬		
Operational decision with LabECO; final report and revision							▬	
Defence								♦

▬ month of the activity; ♦ milestone, from the month shown

References

BAUER, Peter *et al.* The digital revolution of Earth-system science. **Nature Computational Science**, v. 1, n. 2, p. 104–113, 2021.

CAMPOS, Ricardo Martins *et al.* Global assessments of the NCEP ensemble forecast system using altimeter data. **Ocean Dynamics**, v. 70, n. 3, p. 405–419, 2020.

FARINA, Leandro. On ensemble prediction of ocean waves. **Tellus A: Dynamic Meteorology and Oceanography**, v. 54, n. 2, p. 148–158, 2002.

GRAHAM, Susan L.; KESSLER, Peter B.; MCKUSICK, Marshall K. Gprof: A call graph execution profiler. **ACM SIGPLAN Notices**, v. 17, n. 6, p. 120–126, 1982.

HASSELMANN, Klaus *et al.* Measurements of wind-wave growth and swell decay during the Joint North Sea Wave Project (JONSWAP). **Ergänzungsheft zur Deutschen Hydrographischen Zeitschrift, Reihe A**, n. 12, p. 1–95, 1973.

HOFFMANN, Matheus. **ww3-gpu: An open lab for WAVEWATCH III operational optimisation and GPU kernels**. 2026. Disponível em: <https://github.com/h0ffmann/ww3-gpu>. Acesso em: 18 sep. 2026.

IKUYAJOLU, Olawale James *et al.* Porting the WAVEWATCH III (v6.07) wave action source terms to GPU. **Geoscientific Model Development**, v. 16, p. 1445–1458, 2023.

KOLDUNOV, Nikolay V. *et al.* **An ocean model ported by a large language model: Experience and lessons from FESOM2 (Fortran to C to C++/Kokkos)**. arXiv:2606.11356, 2026. Disponível em: <https://arxiv.org/abs/2606.11356>. Acesso em: 1 oct. 2026.

LABECO/UFSC. **ReNOMO – rede nacional de observação e monitoramento oceânico**. 2026. Disponível em: <https://labeco.ufsc.br/renomo/>. Acesso em: 18 sep. 2026.

LINUX KERNEL ORGANIZATION. **Perf: Linux profiling with performance counters**. 2026. Disponível em: <https://perf.wiki.kernel.org/>. Acesso em: 18 sep. 2026.

NOAA-EMC. **WAVEWATCH III source code repository: Develop branch, v7.14**. 2026a. Disponível em: <https://github.com/NOAA-EMC/WW3>. Acesso em: 15 sep. 2026.

NOAA-EMC. **WAVEWATCH IV (WW4) repository**. 2026b. Disponível em: <https://github.com/NOAA-EMC/WW4>. Acesso em: 7 oct. 2026.

NOAA/NWS/NCEP/EMC. **Operational wave models**. 2026. Disponível em: <https://www.emc.ncep.noaa.gov/emc/pages/numerical_forecast_systems/wavemodels.php>. Acesso em: 16 sep. 2026.

THE WAMDI GROUP. The WAM model—a third generation ocean wave prediction model. **Journal of Physical Oceanography**, v. 18, n. 12, p. 1775–1810, 1988.

THE WAVEWATCH III DEVELOPMENT GROUP. **User manual and system documentation of WAVEWATCH III version 6.07**. College Park, MD: NOAA/NWS/NCEP/M 2019. (Tech. Note, 333).

TOLMAN, Hendrik L. A third-generation model for wind waves on slowly vary-

ing, unsteady, and inhomogeneous depths and currents. **Journal of Physical Oceanography**, v. 21, n. 6, p. 782–797, 1991.

TOLMAN, Hendrik L. **The WAVEWATCH III software modernization project: Phase I report**. College Park, MD: NOAA/NWS/NCEP, 2025. (Office Note, 525). Disponível em: <https://doi.org/10.25923/h7j3-1h25>. Acesso em: 15 sep. 2026.

TOLMAN, Hendrik L. **The WAVEWATCH IV project: Phase II report**. College Park, MD: NOAA/NWS/NCEP, 2026. (Office Note, 528). Disponível em: <https://doi.org/10.25923/0wyp-9f39>. Acesso em: 15 sep. 2026.

TROTT, Christian R. *et al.* Kokkos 3: Programming model extensions for the exascale era. **IEEE Transactions on Parallel and Distributed Systems**, v. 33, n. 4, p. 805–817, 2022.

UFSC. **Laboratório de engenharia e ciências oceânicas (LabECO)**. 2026. Disponível em: <https://emc.ufsc.br/portal/laboratorios-2/labeco/>. Acesso em: 18 sep. 2026.

YUAN, Ye *et al.* Towards a real-time modeling of global ocean waves by the fully GPU-accelerated spectral wave model WAM6-GPU v1.0. **Geoscientific Model Development**, v. 17, n. 16, p. 6123–6136, 2024.

Rio de Janeiro, October 1, 2026.

Matheus Hoffmann Fernandes Santos - Student

To be defined, Dr. - Advisor

Pedro Veras Guimarães, Dr. - Co-advisor